
PDDL CODER: AGENTIC PDDL GENERATION FOR LLM-ASSISTED SYMBOLIC PLANNING

A PREPRINT

Veit Laule
Technical University of Berlin
Berlin, Germany
v.laule@campus.tu-berlin.de

Jiangtao Shuai
Technical University of Berlin
Berlin, Germany
jiangtao.shuai@tu-berlin.de

Manfred Hauswirth
Fraunhofer FOKUS & Technical University of Berlin
Berlin, Germany
manfred.hauswirth@tu-berlin.de

Sonja Schimmler
Fraunhofer FOKUS & Technical University of Berlin
Berlin, Germany
sonja.schimmler@tu-berlin.de

ABSTRACT

LLMs remain unreliable for long-horizon planning, often generating logically inconsistent or non-applicable plans. Recent hybrid methods instead translate natural language into the Planning Domain Definition Language (PDDL), allowing symbolic planners to produce verifiable plans. However, existing methods frequently rely on rigid generation pipelines, a partial PDDL definition, or human feedback. Furthermore, their evaluation is hindered by the lack of standardized benchmarks with automated verification. To address these limitations, we present PDDLCoder, an agentic framework for PDDL generation from natural language that iteratively generates, analyzes, and refines planning specifications. We further introduce NL-pddl-gym, a benchmark dataset comprising 711 planning problems across 23 domains with executable gym environments for the automated verification of plan applicability. Experiments on the NL-pddl-gym test set containing 106 problems across 4 held-out domains show that PDDLCoder generates applicable plans for 89.6% of tested planning problems. This improves upon our adaptations of previous PDDL generation methods, which achieved up to 45.3%, and outperforms direct LLM planning approaches, which reached up to 74.5% on the same test set. Our work demonstrates the effectiveness of agentic PDDL generation for planning and establishes a reproducible benchmark for future research on LLM-assisted symbolic planning.

 github.com/vDawgg/PDDLCoder

Keywords AI Planning · Agents

1 Introduction

Automated planning concerns the problem of finding a sequence of actions that transforms an initial state of an environment into one satisfying a desired goal condition [1]. Unlike single-step decision problems, planning requires reasoning over long action sequences, where each action changes the state on which subsequent decisions depend. Therefore, reliable planning requires both an accurate representation of the environment and the ability to reason about the consequences of actions before execution.

With their broad world knowledge and ability to interpret Natural Language (NL), Large Language Models (LLMs) have recently motivated new approaches for automated planning. One intuitive paradigm, *LLM-Planners*, uses LLMs directly to generate action sequences from NL task descriptions [2]. Although such approaches demonstrate promising capabilities, they require the model to simultaneously infer the environment dynamics, reason over long horizons, and maintain action validity. Recent studies show that LLMs often fail in this setting, producing plans containing actions

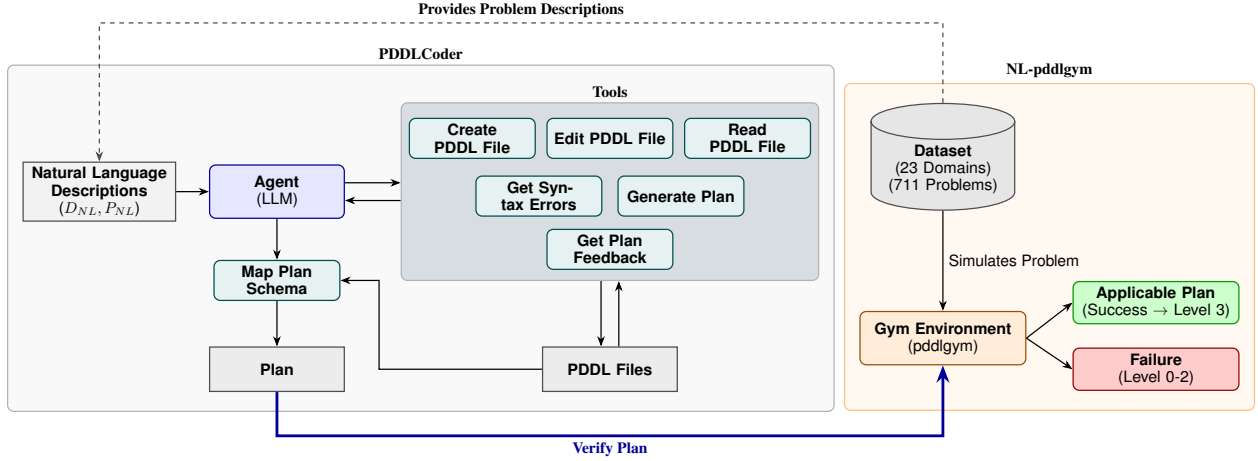


Figure 1: Conceptual overview of the main PDDLCoder pipeline and the interaction with NL-pddlgyM. PDDLCoder receives a set of descriptions from NL-pddlgyM for which the main agent then tries to generate an appropriate set of PDDL files using the available tools. Once this is done, the plan can be mapped to its simulation-ready form and tested for applicability in the matching NL-pddlgyM gym environment.

that violate preconditions or do not reach a goal state [3, 4]. Moreover, without an explicit representation of the planning problem, generated plans lack a reliable basis for validation before execution.

An alternative paradigm, LLM-Formalizers, shifts the role of LLMs from direct planning to formal model construction. These methods translate NL descriptions into formal planning representations, such as the Planning Domain Definition Language (PDDL) [5], which specify actions, preconditions, and effects. A symbolic planner such as Fast Downward (FD) [6] can then generate plans with formal guarantees relative to the generated model, provided that the model faithfully reflects the NL task. We view this formalization process as related to structured code generation, where LLM-based methods have demonstrated strong capabilities [7].

Despite this potential, existing LLM-Formalizers remain limited in three aspects. First, many approaches rely on predefined generation pipelines with fixed refinement procedures [8, 9]. Second, some methods simplify the task by assuming access to partial formal specifications, such as predefined domains [10], or by incorporating human feedback during generation [11]. Third, evaluation often focuses on intermediate properties, such as syntactic validity [12], rather than verifying whether generated specifications produce plans that successfully solve the original task.

To address these limitations, we introduce PDDLCoder, an agentic framework for PDDL generation that iteratively creates, refines, and validates planning specifications through interaction with external tools. Unlike fixed pipelines, PDDLCoder autonomously determines refinement steps and operates zero-shot from NL descriptions and action schemas without human intervention or access to predefined PDDL domains, problems, preconditions, effects or predicates. We further introduce NL-pddlgyM, a benchmark of 711 planning problems across 23 domains with executable environments for directly evaluating plan applicability. Our experiments show that PDDLCoder generates applicable plans for 89.6% of the tested planning problems. This outperforms our adaptations of previous LLM-Formalizers, which achieved up to 45.3%, and LLM-Planners, which reached up to 74.5%.

2 Related Work

2.1 LLM-Planners

With the advent of LLMs, researchers have explored the ability of these models to directly generate plans that solve a given problem. Initial work explored the general reasoning capability of these models by applying LLMs to problems that typically require reasoning through techniques such as Chain of Thought (COT) prompting [13]. Other work [14] utilizes PDDL as a means of unambiguously defining a given task, which is then provided as input to an LLM to generate a plan. This idea was further extended by using LLMs to generate intermediate PDDL formalizations of the problem which are then used by the same model to generate a plan from [15, 16]. Although some success in plan generation from PDDL was reported, LLMs as PDDL-Planners are still outperformed by dedicated planning systems specifically created for solving PDDL problems. In addition to these shortcomings, further work [4], which explored

the long-term planning ability of off-the-shelf LLMs using NL input, found that the models still generally struggle to generate applicable long-horizon plans.

2.2 LLM-Formalizers

In parallel to the exploration of LLM-Planners, researchers started generating formal representations, e.g. PDDL, from NL descriptions, which established planning methods then solve. While LLM-Formalizers can use any number of intermediate formal representations of a given problem such as Python [17], we focus on PDDL as the formal representation in this work. Early work [10] generated PDDL problems from NL descriptions but required a predefined PDDL domain together with matching example pairs in the prompt. Later work loosened these input requirements and additionally provided the model with feedback, such as syntactic checks whose results are incorporated back into the generated PDDL [11].

Subsequent work has improved complementary parts of the construction process. To reduce context-window pressure, NL2PDDL [8] decomposes domain reconstruction into action-level generation steps. VML_PDDL [18] instead applies test-time scaling, selecting from multiple domain candidates before a fixed optimizer–learner refinement schedule, while its problem-synthesis experiments remain separate from domain synthesis. NL2Plan [9] broadens the target to end-to-end domain-and-problem generation from minimal text, but retains a staged extraction procedure followed by one repair pass.

Work	Role	ANL	D+P	AG	AR	VA
PlanBench [19]	Benchmark	✗	–	–	–	✓
Text2World [12]	Benchmark	✗	✗	–	–	✗
LLM+P [10]	Method	✗	✗	✓	✗	✗
LLM-WorldModels [11]	Method	✓	✗	✗	✗	✗
NL2PDDL [8]	Method	✗	✗	✓	✗	✗
NL2Plan [9]	Method	✓	✓	✓	✗	✗
VML_PDDL [18]	Method	✓	✗	✓	✗	✗
NL-pddlgy (ours)	Benchmark	✓	✓	–	–	✓
PDDLCoder (ours)	Method	✓	✓	✓	✓	✓

Table 1: Method and benchmark boundaries. Abstract NL (ANL) means that task descriptions are abstract and full predicate descriptions are not supplied; Domain and Problem (D+P) means joint domain-and-problem generation; Autonomous Generation (AG) means that neither human feedback nor oracle environments are needed during generation; Adaptive Refinement (AR) means that the method is not limited to rigid refinement schedules; Verified Applicability (VA) means that an output plan is executed against task dynamics, rather than assessed only by syntax, structural similarity, or agreement with a reference. Dashes denote a non-applicable method property.

These developments improve decomposition, candidate selection, and task coverage, but Table 1 highlights two remaining distinctions. First, their refinement schedules are specified in advance rather than selected in response to the current failure. PDDLCoder addresses this workflow limitation by treating syntax errors, unsolvable PDDL models, and solvable models whose plans conflict with the task semantics as different states requiring different tools. To achieve this, the agent chooses its next action from the evolving workspace instead of applying the same repair sequence to every instance. Second, formal validity or symbolic solvability alone does not establish that a generated plan solves the task expressed in NL. We address this separate evaluation gap with NL-pddlgy, which executes the final mapped plan against held-out task dynamics, as described next.

2.3 Benchmarks for Language-to-PDDL Formalization

Existing benchmarks emphasize different stages of the formalization-to-execution pipeline. Planetarium [20] provides a large collection of text-to-PDDL problem pairs and a semantic-equivalence test, but assumes fixed domains and is limited to three classical domains. Text2World [12] broadens domain coverage and evaluates generated domain models with syntactic validity, solvability, structural, and component-level metrics, but does not verify plan-applicability, supplies predicate descriptions and does not require a matching problem file.

PlanBench [19] shifts attention to the downstream stage: it uses planner- and validator-based checking to evaluate direct planning and reasoning about change. Its few-shot prompts specify the lifted actions, preconditions, and effects, leaving domain construction outside the task. Our NL-pddlgy connects formalization to this downstream evaluation. It provides a high-level task description, action names, objects, initial states, and goals. Separately, the

system receives the minimal target action schema S needed to map generated plans to the interface of the target environment. The model must infer the withheld predicates, preconditions, effects, and any additional parameters needed by the generated PDDL. This design follows the argument of [21] that expert-crafted formalization hints both limit scalability and bias the interpretation of inherently ambiguous NL descriptions. The resulting plan is then mapped and executed in a corresponding gym environment. NL-pddlgyim thus combines low-specification joint domain and problem formalization with plan applicability evaluation while remaining complementary to Text2World’s domain-model analysis and PlanBench’s broader planning curriculum.

3 Approach

We study joint domain-and-problem formalization to generate plans for fully observable and deterministic classical planning problems [22]. Given abstract NL descriptions of a planning domain and a problem, denoted by D_{NL} and P_{NL} , respectively, PDDLCoder uses an LLM and symbolic feedback tools to construct a PDDL domain D and problem P (see Section 3.2), from which a classical planner derives a plan π . The generation process receives neither human corrections nor feedback from the held-out gym environment.

For the final applicability evaluation we do not consider the original π resulting from a given D and P , but a mapped version π_m , which matches the action schema S and object set O expected in the gym environment. S contains actions with the set of parameters that are deemed absolutely necessary to deterministically carry out an action in each domain as in the original pddlgyim implementation [23]. In S , each schema is specified using only its action and parameter names, restricting the amount of domain-specific information provided. For example, a ‘move’ operator in PDDL is often parameterized with a *from* location and a *to* location. While it makes sense to supply the original *from* location in PDDL, as this easily allows for removing the old location from the current set of facts, an agent moving to a new location does not need this information, as it only needs to know which new location to move to [23]. This simplified ‘move’ action would then be given as ‘move(?to)’ in S . By working with this minimal set of parameters for each action, we allow our approach to freely choose the parameters that are used in the generated PDDL while assuring that we can later test the applicability of π_m in the gym environments without encountering issues due to the arity of the generated actions. To generate an appropriate π_m from a given π we use an LLM that we call *MapAgent* which receives, S , O and the generated D and π to generate the matching π_m . We leave probabilistic domains and other extensions to PDDL for future work.

3.1 NL-pddlgyim

We introduce NL-pddlgyim, a benchmark for generating both PDDL domain and problem files from NL descriptions and evaluating the resulting plans through execution. It comprises 711 problems across 23 domains, with a median of 30 problems per domain and between 3 and 310 objects per problem instance, and pairs each (D_{NL}, P_{NL}) input with an executable environment. Our experiments reserve four complete domains as a held-out test set and the remaining domains form training and validation splits for future method development, with no test domain appearing in either split. The executable environments are built with pddlgyim, which provides gym interfaces for PDDL domains and problems [23]. For each generated π_m the evaluator applies the plan’s actions sequentially, and records success only if all actions could be applied, meaning their preconditions were satisfied, and the terminal state satisfies the task goal. NL-pddlgyim therefore supplies a task-level applicability oracle rather than relying only on syntax checks or symbolic solvability. Furthermore, we do not make any claims regarding the semantic equivalence of the generated and ground-truth PDDL. Due to the abstract definition of the planning problem in NL and the explicit omission of predicates from this description, NL-pddlgyim allows for multiple valid PDDL encodings. We therefore limit the benchmark to testing the applicability of the plans resulting from the PDDL files for a given problem.

The underlying PDDL domains and problems were drawn from the original pddlgyim implementation and the classical-domains repository¹, complemented by three domains and their problems created specifically for this work. We restrict all domains to the requirements *typing*, *strips*, *disjunctive-preconditions*, *conditional-effects*, *negative-preconditions* and *equality*. To construct the language side of the benchmark, we manually authored all 23 domain descriptions and two to three seed problem descriptions per domain, then used DeepSeek v4 Flash [24] to generate the descriptions for the remaining problems, instructing the model to follow the seed patterns while preserving the objects, initial state, and goal of the source PDDL problem. Each domain description summarizes the task setting and lists the available action names, while each problem description specifies the objects, initial state, and goal. To preserve the abstract formalization setting, D_{NL} and P_{NL} omit predicate signatures, action parameters, preconditions, and effects. In addition to this, the dataset also contains an object list O and action schema list S needed to map generated plans π to their simulation-ready form π_m . An example of D_{NL} and P_{NL} for the Ring and Peg domain from the test set can be seen in Figure 2.

¹classical-domains GitHub repository <https://github.com/AI-Planning/classical-domains>

Ring and Peg Domain Description	Exemplary Ring and Peg Problem Description
<pre>## Domain description A robot arm is tasked to sort a set of colored rings on a set of pegs of the same colors. The arm can be used to pick up a ring from a peg, move a ring to another peg and place the ring on the other peg. Rings are already placed on pegs and can only be placed on pegs. To pick up a ring at a peg, the robot first has to move to the pegs position. The actions available to the robot are: - move - Moves the robot to a specified position - pick - Closes the robot arms gripper at the current position - place - Opens the robot arms gripper at the current position</pre>	<pre>## Problem description There are 5 colored pegs with the following colors: red, green, blue, pink, yellow. The pegs are all named in the format <color>_peg. Additionally there are 3 colored rings: red, green and blue. Similarly to the pegs, the rings are named <color>_ring. The robot arm starts out in a default position, while the rings start in the following positions: - red_ring - pink_peg - green_ring - yellow_peg - blue_ring - red_peg The goal is to transfer all rings to the pegs of their color.</pre>

Figure 2: Exemplary D_{NL} and P_{NL} of the Ring and Peg domain from the NL-pddl gym test set

3.2 PDDLCode

Figure 1 depicts the control flow and system architecture of PDDLCode. Orchestrated via the DSPy [25] implementation of ReAct [26], the agent receives the natural language descriptions (D_{NL} , P_{NL}) and general PDDL guidelines to iteratively construct the domain D and problem P . The tools the agent has access to can be broadly divided into two categories: the first allows for the creation, reading, and editing of PDDL files, while the second allows the agent to gather feedback on the current state of the generated files. The agent continuously interacts with these tools until it explicitly signals completion or reaches a hard limit of 50 iterations, which prevents context window exhaustion. Following loop termination, the system validates the syntax of D and P and attempts to generate a plan π . If successful, π is mapped to π_m by the MapAgent and saved. Otherwise, a syntax error or planning failure is recorded.

3.3 Create Read Update (CRU) Tools

The Create Read Update (CRU) tools we use in the pipeline are similar to part of the code-editing interfaces outlined in [7], with the main difference being that the operations in our approach are specifically limited to working with PDDL files. While the creation and reading tools directly manipulate files, the edit tool first applies modifications in memory and validates them using the VAL parser [27]. If syntax errors are detected, the edit is rejected and reported back. Otherwise, the updated file contents are returned.

3.4 Feedback Tools

The tools in this category provide feedback on the quality of the generated PDDL using external programs, such as VAL [27] or FD [6], allowing the agent to check syntax, generate plans, or assess plan applicability.

3.4.1 Syntax Check Tools

The syntax tools leverage VAL to report errors with line numbers and file contexts after the initial creation of D or P . To complement VAL, we also provide a tool running the translation phase of FD [6]. While VAL offers detailed error diagnostics and line numbers, FD’s translation step acts as a stricter syntax check required before plan generation.

3.4.2 Plan Generation Tools

Once a set of PDDL files has been created, the agent can call a tool to verify if D and P can be used to generate a plan, returning either π or a planning-failure report. For the plan generation, we use the *lama-first* configuration of FD, which, while not guaranteed to produce optimal plans, quickly returns the first plan it finds enabling fast feedback loops.

To support parallel execution, we cap the memory of each FD run at 4 GB and additionally limit execution time to 1 minute. When planning fails due to unsolvability, we parse a curated subset of FD’s verbose output to extract what we consider to be the most actionable data. This includes the number of reachable grounded PDDL atoms, grounded actions, actions removed due to impossible preconditions and effects, and information on the satisfiability of the goal state. In cases where planning fails due to time- or memory-limits, we simply return a message stating this information.

3.4.3 Plan Feedback Tool

Once π is generated, the agent can invoke a feedback tool to evaluate its task applicability, inspired by iterative refinement methods [28]. Rather than evaluating the raw plan π , this tool prompts a dedicated *PlanAgent* using only the natural language descriptions (D_{NL} , P_{NL}), the target action schema S , and the mapped plan π_m . We intentionally withhold the generated D and P to prevent the PlanAgent from over-indexing on the syntactic design of the formal representations, ensuring its feedback remains focused on the semantic feasibility of the action sequence. The PlanAgent parses π_m to identify violations of explicit or implicit constraints derived from D_{NL} . For example, in the *Ring and Peg* domain, where a set of rings needs to be transferred between a set of pegs, LLMs frequently generate plans that try to move a ring while it is stacked underneath another ring on the same peg. The PlanAgent can identify this implicit physical violation in the mapped plan and provide actionable guidance to the main agent to correct the underlying PDDL.

4 Evaluation

We evaluate PDDLCoder on the test set of the NL-pddlgyim dataset. This subset of the dataset consists of 4 domains not included in the remaining dataset splits, which are intended for optimization approaches. The test set includes a domain (Ring and Peg) that we specifically created for this dataset, which to our knowledge has not been publicly available prior to these experiments. The remaining domains are the established classical planning domains Hanoi, Elevator and Satellite. The total number of problems in this test set amounts to 106 problem variations with varying numbers of objects, resulting in plans ranging between 2 and 557 steps.

To account for the influence of different LLMs and the impact of the different tools we designed for PDDLCoder, we evaluate the approach across a set of open-weight models and four ablations. For these ablations we either remove all feedback tools, only the plan feedback tool, or the plan mapping step for the feedback tool. This set of ablations is complemented by a rigid version of PDDLCoder to analyze the impact of the methods execution-style. In addition to the ablations of our approach, we also include comparisons with adaptations of previous LLM-Formalizer and LLM-Planner methods. The set of LLM-Formalizer methods we consider consists of NL2Plan [9] and an adapted version of VML_PDDL [18], where we use LLM-generated scores for the BoN selection instead of selecting via log-probs as in the original paper due to restrictions of the DeepSeek API we used for the experiments. For comparison with LLM-Planner methods, we evaluate the self-validator variant of ISR-LLM, which uses few-shot prompts from the Blocksworld domain [15], and a COT [13] prompting approach on the same test set. All of the approaches we compare with PDDLCoder were executed in their best-performing configurations including original prompts, candidate counts, iteration limits and stopping criteria as specified in the original papers. To ensure a fair comparison with these approaches, we employ the same mapping approach for the generated plans as used in PDDLCoder. Each model, problem and approach configuration was evaluated once. The reported results therefore do not quantify sampling variance between different runs.

We separately verified the quality of the MapAgent approach by running the MapAgent on the plans generated from the ground-truth PDDL files from the NL-pddlgyim test set. After applying each of the generated plans to the appropriate gym environments, we found that the mapped plans yielded a 98.1% success rate in solving the problem given a correct initial unmapped plan.

Level	Description
0	No syntactically valid PDDL.
1	Syntactically valid PDDL but no plan found.
2	Non-applicable plan.
3	Plan solves the given problem.

Table 2: Levels for PDDL generation.

To evaluate this varied set of approaches we use a metric we created for the PDDL generation task. This metric is designed to measure the capability of a PDDL generation approach along a set of four distinct levels, denoting different

stages of increasing difficulty that need to be passed during PDDL generation, which are described in more detail in Table 2. Generally, level 0 indicates inability to generate syntactically valid PDDL, meaning the generated PDDL was rejected by the VAL parser or FD translation layer, and level 3 indicates the ability to generate PDDL that produces applicable plans solving a given problem. Notably, a plan is applicable iff each action is applicable in the given sequence and the execution of the terminal action satisfies the goal. In addition to the LLM-Formalizer approaches we evaluate here, we also evaluate the LLM-Planner approaches using this metric. However, here the metric only accounts for the generation of non-applicable or applicable plans (levels 2 and 3) as no PDDL is generated using these methods.

4.1 Results Across Models

We initially present the results of our approach when applied to a variety of different open-weight models, to verify whether PDDLCoder scales to different LLMs.

Model	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
DeepSeek v4 Flash	0	2	9	95	90	75	93.3	94.4	172978.8	234922.6	32809.4	32990.4
Gemma 4 31b	8	2	31	65	20	85	63.3	69.4	411577.6	404900.1	15453.8	12257.4
gpt-oss-120b	26	3	21	56	80	55	46.7	41.7	307318.1	424007.3	27589.4	29793.0
Qwen 3.6 35b A3b	45	1	49	11	30	5	6.7	5.6	129916.1	251746.6	37574.6	40290.6
GLM 4.7 Flash	66	17	22	1	5	0	0	0	762909.5	721951.2	35162.3	28381.5
Llama 4 Scout 17b 16e	99	3	4	0	0	0	0	0	521045.1	154822.6	13961.4	6312.5

Table 3: Performance of PDDLCoder across different open-weight LLMs on the NL-pddl-gym test set. Also showing the success rates (i.e. reached level 3) for the Elevator (El), Hanoi (Ha), Ring and Peg (RP) and Satellite (Sa) domains from the test set. Additionally includes the average number of input (N-In) and output (N-Out) tokens together with standard deviation (S-In and S-Out).

In the case of DeepSeek v4 Flash, the model was used through OpenRouter due to the possibility of executing multiple experiments in parallel, while the other models were hosted locally using vLLM [29]. Looking at the results in Table 3 we can see that, although a portion of the models performs well, the effectiveness of PDDLCoder is dependent on the ability of the LLMs to generate usable plan feedback and syntactically valid PDDL. Amid runs that produce syntactically valid PDDL, relatively few terminate at level 1. This suggests that planner solvability is less frequently a bottleneck than producing syntactically valid files or ensuring that the resulting plan is applicable.

Among the tested models, DeepSeek v4 Flash [24] performs the best, generating PDDL resulting in applicable plans for 89.6% of the problems, followed by the smaller Gemma 4 31b [30] and gpt-oss-120b [31]. The Gemma model performs surprisingly well for its size when compared to the 258 billion parameter DeepSeek model, and is able to generate syntactically valid PDDL in 92.5% of the cases. However, the model frequently fails to point out issues with generated plans, leading to 29.2% not being applicable versus 8.5% of non-applicable plans among the tested problems for the DeepSeek model. These results indicate that the quality of plan feedback is strongly model-dependent, although we cannot clearly state whether this variation is caused by model scale, training, or instruction-following ability. More than this, some models like Qwen 3.6 35b A3b [32] and gpt-oss-120b frequently fail to adhere to the expected response format, resulting in no PDDL being generated for a given problem. In other instances, the LLMs fail to generate solvable D and P for the Hanoi domain, as they are unable to adhere to the requirement stated in D_{NL} that the destination of the move action must be a peg. Here, the LLMs frequently start out with generating PDDL that does not fit this constraint and are unable to generate PDDL that is correctly aligned before the budget is exhausted. Most notably, other models like GLM 4.7 Flash [33] and Llama 4 Scout 17b 16e [34] are largely unable to generate syntactically valid PDDL indicated by the models only generating PDDL without syntax issues in 37.7% or even 6.6% of the cases respectively. Specifically, the models fail to apply any fixes to the syntactically invalid PDDL generated by them and frequently try to apply the same invalid edit until the iteration budget is exhausted. Seeing this, a major bottleneck for PDDLCoder is the ability of an LLM to generate syntactically valid PDDL as this forms the basis for all later stages in the PDDL generation process.

4.2 Comparison with Previous Approaches

Table 4 compares PDDLCoder with adaptations of previous LLM-Formalizers and LLM-Planners. PDDLCoder achieves a level-3 success rate of 89.6%, compared with 45.3% for NL2Plan, 34.9% for VML_PDDL, 74.5% for COT, and 69.8% for ISR-LLM. PDDLCoder achieves this performance while requiring a lower number of output tokens on

average than other LLM-Formalizers, although it consumes substantially more input tokens due to long tool outputs with full PDDL files or plans.

Approach	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
PDDLCoder	0	2	9	95	90	75	93.3	94.4	172978.8	234922.6	32809.4	32990.4
VML_PDDL	52	4	13	37	75	10	33.3	27.8	83338.5	63977.7	123071.6	26272.6
NL2Plan	34	5	19	48	70	40	33.3	44.4	149727.6	70625.3	44023.2	26275.4
ISR-LLM	-	-	32	74	80	40	83.3	69.4	13467.3	10841.8	22758.5	19680.3
COT	-	-	27	79	90	60	96.7	55.6	2473.7	2603.0	7410.1	8658.1

Table 4: Performance of PDDLCoder compared with adapted versions of previous LLM-Formalizers (VML_PDDL and NL2Plan) and LLM-Planners (ISR-LLM and COT). All approaches were tested on the NL-pddlgyim test set with DeepSeek v4 Flash.

The improvements over VML_PDDL [18] might be due to the lack of feedback mechanisms, which are present in PDDLCoder. Although the model is also able to iteratively refine the generated PDDL it cannot externally verify that the generated PDDL is actually syntactically correct or solvable, leading to 49% of the generated problems not passing syntax checks.

In contrast to this, NL2Plan [9] offers even more feedback mechanisms to the model than PDDLCoder but might do so in a too rigid and narrow manner. Specifically, the pipeline fails to generate syntactically valid PDDL for 34 of the planning problems from the test set, which most frequently stems from actions using non-existent predicates or ill-defined action names. This is partially due to syntactically incorrect action names which cannot be changed after their initial generation. While this approach also incorporates a form of plan feedback during generation, the feedback is only ever generated once. This assumes that the initial feedback is perfect and that the issue can be resolved with this feedback alone, which is often not the case, as suggested by the large number of non-applicable plans. In contrast to this, PDDLCoder allows for multiple calls of the plan feedback (within the total budget), allowing the model to iteratively verify that its changes resolved issues that were found by the PlanAgent.

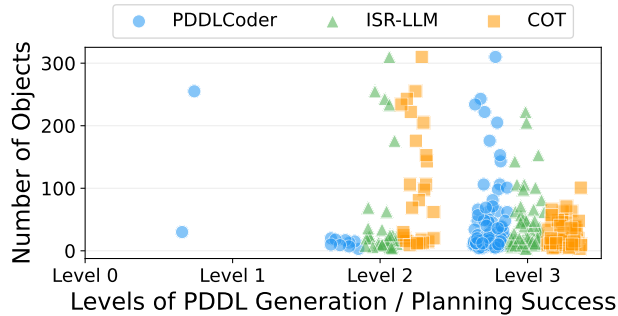


Figure 3: Relationship between object count and performance compared across PDDLCoder, COT, and ISR-LLM.

Surprisingly, the LLM-Planner approaches we tested outperform all tested LLM-Formalizer approaches except PDDLCoder when used with DeepSeek v4 Flash on the NL-pddlgyim test set, although this ranking does not always transfer to other models in our testing. Even though ISR-LLM [15] uses a more sophisticated pipeline design when compared to COT [13] both LLM-Planner approaches perform comparably. Notwithstanding its overall good performance, COT still struggles with generating applicable plans for complex planning problems as already noted in previous work that looked into the planning ability of LLMs [35, 4]. This becomes apparent when looking at Figure 3, where the number of objects, which we use as a proxy to the problem’s complexity similarly to [35], is compared with the performance of the approaches. Here, COT fails to generate applicable plans that involve more than 100 objects, due to missing required actions or the refusal to generate plans at all in sufficiently complicated problem instances. However, a similar cut-off cannot be found for ISR-LLM, although this method still frequently fails to adhere to the required output format for the generated plan. On the same test set, PDDLCoder is able to generate applicable plans for the tested problem instance with the highest object count. Nevertheless, unlike PDDLCoder, the direct LLM-Planner approaches do not produce a reusable formal model from which a symbolic planner can deterministically regenerate plans.

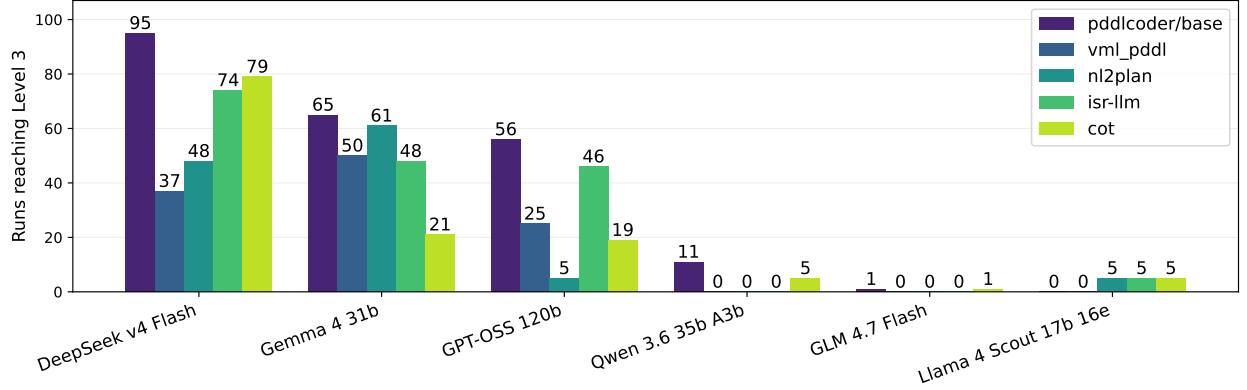


Figure 4: Performance (number of runs reaching level 3) across open-weight models and adaptations of previous LLM-Planner and LLM-Formalizer approaches. ISR-LLM was not evaluated with Qwen 3.6 35b A3b because the model does not support the intermediate system prompts required by the method. The corresponding zero-height bar denotes missing results rather than zero successful instances.

Finally, when looking at the performance of each of the distinct approaches across the open-weight models in Figure 4, we can see that PDDLCoder continues to be the strongest approach, with the exception of the results of GLM 4.7 and Llama 4 Scout 17b 16e. Performance varies substantially across models, and the relative ranking of the previous approaches is not consistent, highlighting the importance of model selection. Nevertheless, PDDLCoder achieves the highest or tied-highest level-3 performance for all models except Llama 4 Scout 17b 16e.

4.3 Ablation Results

As can be seen in Table 5 the tools we designed as part of PDDLCoder all positively influence the syntactic validity and solvability of the generated PDDL and the applicability of generated plans. Specifically, removing all feedback tools except for the CRU tools from the pipeline results in PDDLCoder generating syntactically invalid PDDL for 11% of the tested problems, suggesting that the external feedback contributes to syntactic reliability. Furthermore, we can see that the addition of the PlanAgent increases the number of applicable plans generated for the test set. This is due to the approach often generating plans that do not respect logical constraints, such as stacking constraints that are not correctly inferred for the Ring and Peg domain described earlier, which can be pointed out by the PlanAgent. However, this increase in the number of applicable plans is also accompanied by a $2.74x$ increase in the number of consumed output tokens, which is the largest among the tested agentic PDDLCoder ablations. Additionally, we can see that while the usage of π_m during plan feedback provides a noticeable improvement, it does not constitute a similar impact as the PlanAgent which results in a 20.8 pp. decrease in level 3 outcomes when removed from the full pipeline, while the additional switch from π_m to π for plan feedback only results in a further 7.5 pp. decrease.

Ablation	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
PDDLCoder	0	2	9	95	90	75	93.3	94.4	172978.8	234922.6	32809.4	32990.4
Rigid Execution	19	2	25	60	50	60	46.7	66.7	39319.1	31494.2	57247.8	43709.1
Without MapAgent	7	1	11	87	90	70	90	77.8	320124.2	355922.2	41477.0	31389.1
Without PlanAgent	3	2	28	73	90	85	40	72.2	74907.8	65274.9	15127.5	11861.2
CRU Only	12	7	19	68	95	60	36.7	72.2	48243.1	41198.7	18176.0	11773.6

Table 5: Performance of PDDLCoder compared with a rigid version of the pipeline and ablations where the plan mapping step was removed from the plan feedback and the feedback works on the unmapped π , plan feedback was removed altogether and where no tools except those for CRU are available. All variants were tested on the NL-pddl gym test set with DeepSeek v4 Flash.

Furthermore, we also created a variant of the PDDLCoder approach where the same tools and agent are available, but the pipeline is orchestrated in a step-by-step sequence. Specifically, the pipeline starts by first generating the domain, fixing eventual syntax errors in a loop of up to 5 iterations and goes on to generating the problem with a similar syntax feedback loop. Once a set of syntactically valid PDDL files has been generated, the system tries to generate a plan and

starts another feedback loop, where the model is fed the specific issues with instructions to fix the domain and problem files, in cases where the current pair of PDDL files is unsolvable. Once a plan has been generated, it is mapped by the MapAgent and fed into the PlanAgent which generates appropriate feedback. This feedback is then fed back to the start of the pipeline together with the already created PDDL files with instructions to fix any possible issues. This feedback loop repeats for 5 iterations at which point the pipeline completes, resulting in a minimum of 10 and a maximum of 102 total model calls depending on the number of executed feedback loops. Although many alternative rigid pipelines could be designed and the interaction budgets are not strictly matched, the result provides preliminary evidence that the adaptive tool selection of PDDLCode contributes beyond available tools alone. Specifically, the rigid pipeline variant is outperformed by our proposed pipeline implementation in all tested domains. While the rigid implementation requires significantly fewer input tokens, as the previous interaction history is not fed into the model on each call, the agentic variant only requires 57.3% of the average number of output tokens consumed by the rigid variant, as the model can apply more focused edits instead of having to regenerate the entire PDDL file for each change.

5 Conclusion

In this work, we introduce PDDLCode, an autonomous, agentic LLM-Formalizer comparable to modern coding agents. By iteratively creating, analyzing, and refining planning specifications, PDDLCode outperforms our adaptations of both existing LLM-Formalizers and direct LLM-Planners in generating executable, applicable plans on the NL-pddl-gym test set. To support rigorous evaluation in this domain, we also present NL-pddl-gym, a benchmark of 711 abstract planning problems that verifies the applicability of plans generated for abstract domain and problem descriptions in executable gym environments. While PDDLCode demonstrates strong zero-shot capabilities, the PlanAgent currently relies on sufficient reasoning capacity of LLMs to correctly identify and resolve logical inconsistencies, while the overall pipeline quality depends on the LLM’s general ability to generate syntactically valid and solvable PDDL. To address this, future work will explore automated prompt optimization utilizing the training and validation splits of NL-pddl-gym. Ultimately, we believe this agentic framework and execution-based benchmark establish a strong, reproducible foundation for the next generation of LLM-assisted symbolic planning.

6 Acknowledgments

This study was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under the National Research Data Infrastructure – NFDI 2/2 ‘NFDI4Cat – NFDI für Wissenschaften mit Bezug zur Katalyse’ – 441926934.

References

- [1] R. E. Fikes and N. J. Nilsson, “Strips: A new approach to the application of theorem proving to problem solving,” *Artificial intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [2] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, D. Kalashnikov, S. Levine, Y. Lu, C. Parada, K. Rao, P. Sermanet, A. T. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, N. Brown, M. Ahn, O. Cortes, N. Sievers, C. Tan, S. Xu, D. Reyes, J. Rettinghouse, J. Quiambao, P. Pastor, L. Luu, K.-H. Lee, Y. Kuang, S. Jesmonth, N. J. Joshi, K. Jeffrey, R. J. Ruano, J. Hsu, K. Gopalakrishnan, B. David, A. Zeng, and C. K. Fu, “Do as i can, not as i say: Grounding language in robotic affordances,” in *6th Conference on Robot Learning (CoRL 2022)* (K. Liu, D. Kulic, and J. Ichnowski, eds.), vol. 205 of *Proceedings of Machine Learning Research*, pp. 287–318, PMLR, 2023.
- [3] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” in *39th International Conference on Machine Learning (ICML 2022)* (K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, eds.), vol. 162, pp. 9118–9147, PMLR, 2022.
- [4] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, “On the planning abilities of large language models - a critical investigation,” in *37th Conference on Neural Information Processing Systems (NeurIPS 2023)* (A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds.), vol. 36, pp. 75993–76005, Curran Associates, Inc., 2023.
- [5] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins, “Pddl—the planning domain definition language,” tech. rep., Yale Computer Science, 1998.
- [6] M. Helmert, “The Fast Downward planning system,” *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.

- [7] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *38th Conference on Neural Information Processing Systems (NeurIPS 2024)* (A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, eds.), vol. 37, pp. 50528–50652, Curran Associates, Inc., 2024.
- [8] J. Oswald, K. Srinivas, H. Kokel, J. Lee, M. Katz, and S. Sohrabi, “Large language models as planning domain generators,” in *34th International Conference on Automated Planning and Scheduling (ICAPS 2024)*, vol. 34, pp. 423–431, AAAI Press, 2024.
- [9] E. Gestrin, M. Kuhlmann, and J. Seipp, “NL2Plan: Robust LLM-driven planning from minimal text descriptions,” in *Workshop on Human-Aware and Explainable Planning at the 34th International Conference on Automated Planning and Scheduling (ICAPS 2024)*, AAAI Press, 2024.
- [10] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “Llm+p: Empowering large language models with optimal planning proficiency,” 2023.
- [11] L. Guan, K. Valmeekam, S. Sreedharan, and S. Kambhampati, “Leveraging pre-trained large language models to construct and utilize world models for model-based task planning,” in *37th Conference on Neural Information Processing Systems (NeurIPS 2023)* (A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds.), vol. 36, pp. 79081–79094, Curran Associates, Inc., 2023.
- [12] M. Hu, T. Chen, Y. Zou, Y. Lei, Q. Chen, M. Li, Y. Mu, H. Zhang, W. Shao, and P. Luo, “Text2world: Benchmarking large language models for symbolic world model generation,” in *Findings of the Association for Computational Linguistics (ACL 2025)* (W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, eds.), pp. 26043–26066, Association for Computational Linguistics, 2025.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *36th Conference on Neural Information Processing Systems (NeurIPS 2022)* (S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, eds.), vol. 35, pp. 24824–24837, Curran Associates, Inc., 2022.
- [14] T. Silver, V. Hariprasad, R. S. Shuttlesworth, N. Kumar, T. Lozano-Pérez, and L. P. Kaelbling, “Pddl planning with pretrained large language models,” in *Foundation Models for Decision Making Workshop at the 36th Conference on Neural Information Processing Systems (NeurIPS 2022)*, Curran Associates, Inc., 2022.
- [15] Z. Zhou, J. Song, K. Yao, Z. Shu, and L. Ma, “Isr-llm: Iterative self-refined large language model for long-horizon sequential task planning,” in *2024 International Conference on Robotics and Automation (ICRA 2024)*, pp. 2081–2088, IEEE, 2024.
- [16] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. P. Saldyt, and A. B. Murthy, “Position: LLMs can’t plan, but can help planning in LLM-modulo frameworks,” in *41st International Conference on Machine Learning (ICML 2024)* (R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, eds.), vol. 235 of *Proceedings of Machine Learning Research*, pp. 22895–22907, PMLR, 21–27 Jul 2024.
- [17] P. P. Kagitha, B. Sun, I. Desai, A. Zhu, C. Huang, M. Li, Z. Li, and L. Zhang, “Unifying inference-time planning language generation,” in *Findings of the Association for Computational Linguistics: ACL 2026* (M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, eds.), (San Diego, California, United States), pp. 8531–8574, Association for Computational Linguistics, July 2026.
- [18] Z. Yu, Y. Yuan, T. Z. Xiao, F. F. Xia, J. Fu, G. Zhang, G. Lin, and W. Liu, “Generating symbolic world models via test-time scaling of large language models,” *Transactions on Machine Learning Research*, 2025.
- [19] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati, “Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change,” in *37th Conference on Neural Information Processing Systems (NeurIPS 2023)* (A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds.), pp. 38975–38987, Curran Associates, Inc., 2023.
- [20] M. Zuo, F. P. Velez, X. Li, M. Littman, and S. Bach, “Planetarium: A rigorous benchmark for translating text to structured planning languages,” in *2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2025)* (L. Chiruzzo, A. Ritter, and L. Wang, eds.), pp. 11223–11240, Association for Computational Linguistics, 2025.
- [21] S. Huang, N. Lipovetzky, and T. Cohn, “Planning in the dark: Llm-symbolic planning pipeline without experts,” in *39th AAAI Conference on Artificial Intelligence (AAAI 2025)*, pp. 26542–26550, AAAI Press, 2025.
- [22] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: theory and practice*. Elsevier, 2004.

- [23] T. Silver and R. Chitnis, “Pddl gym: Gym environments from pddl problems,” in *Workshop on Bridging the Gap Between AI Planning and Reinforcement Learning at the 30th International Conference on Automated Planning and Scheduling (ICAPS 2020)*, AAAI Press, 2020.
- [24] Deepseek AI, “Deepseek-v4: Towards highly efficient million-token context intelligence,” 2026.
- [25] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, “Dspy: Compiling declarative language model calls into self-improving pipelines,” in *12th International Conference on Learning Representations (ICLR 2024)*, ICLR, 2024.
- [26] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *11th International Conference on Learning Representations (ICLR 2023)*, ICLR, 2023.
- [27] R. Howey, D. Long, and M. Fox, “Val: automatic plan validation, continuous effects and mixed initiative planning using pddl,” in *16th International Conference on Tools with Artificial Intelligence (ICTAI 2004)*, pp. 294–301, IEEE, 2004.
- [28] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *37th Neural Information Processing Systems (NeurIPS 2023)* (A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds.), vol. 36, pp. 46534–46594, Curran Associates, Inc., 2023.
- [29] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *29th ACM SIGOPS Symposium on Operating Systems Principles (SOSP 2023)*, 2023.
- [30] Google Deepmind, “Gemma 4.” <https://deepmind.google/models/gemma/gemma-4/>, 4 2026. Accessed: 2026-07-29.
- [31] OpenAI, “gpt-oss-120b & gpt-oss-20b model card,” 2025.
- [32] Qwen Team, “Qwen3.6-35B-A3B: Agentic coding power, now open to all.” <https://qwen.ai/blog?id=qwen3.6-35b-a3b>, April 2026. Accessed: 2026-07-29.
- [33] GLM Team, “Glm-4.5: Agentic, reasoning, and coding (arc) foundation models,” 2025.
- [34] Meta AI, “The Llama 4 herd: The beginning of a new era of natively multimodal ai innovation.” <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, April 2025. Accessed: 2026-07-29.
- [35] P. Shojaei, S. I. Mirzadeh, K. Alizadeh, M. Horton, S. Bengio, and M. Farajtabar, “The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity,” in *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, vol. 39, Curran Associates, Inc., 2025.

A Experimental Settings

All experimental results were gathered on two systems:

1. Laptop running Ubuntu 24.04.4 with an Intel Core I7-12800H and 64 GB of RAM.
2. GPU server running Ubuntu 24.04.4 with an Intel Xeon Platinum 8480C CPU, 100 GB of RAM and 4 NVIDIA H100 GPUs.

System 1 was used for gathering all results using DeepSeek v4 Flash and system 2 was used to gather the results using the other models. While the DeepSeek model was queried through OpenRouter² its temperature was set to 1.0 and Top P of 1.0 according to the original authors specifications. All other models were hosted locally using vLLM with the sampling parameters set according to the original model authors' specifications, which are listed in Table 6.

Model	Temperature	Top K	Top P
Gemma 4 31b	1.0	64	0.95
gpt-oss-120b	1.0	0	1.0
Qwen 3.6 35b A3b	1.0	20	0.95
GLM 4.7 Flash	1.0	0	1.0
Llama 4 Scout 17b 16e	0.6	0	0.9

Table 6: Sampling parameters for all locally hosted models from the experiments.

B Supplemental Results

Note that while performance levels (L0–L3) are reported comprehensively for all models, token consumption statistics for ISR-LLM are omitted in the supplemental tables due to a logging limitation during the local vLLM execution. This does not impact the primary applicability and success rate metrics.

Approach	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
PDDLCoder	8	2	31	65	20	85	63.3	69.4	411577.6	404900.1	15453.8	12257.4
VML_PDDL	36	0	20	50	60	65	6.7	63.9	55211.3	34349.9	42909.2	19945.6
NL2Plan	6	2	37	61	70	75	10	80.6	128786.3	47703.0	13668.6	6832.4
ISR-LLM	–	–	58	48	85	25	16.7	58.3	–	–	–	–
COT	–	–	85	21	0	0	70	0	3946.9	4380.1	3336.4	4808.6

Table 7: Performance of Gemma 4 31b across PDDLCoder and our adaptations of previous LLM-Formalizers (VML_PDDL and NL2Plan) and LLM-Planners (ISR-LLM and COT) on the NL-pddlgyim test set.

Approach	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
PDDLCoder	26	3	21	56	80	55	46.7	41.7	307318.1	424007.3	27589.4	29793.0
VML_PDDL	73	0	8	25	35	20	26.7	16.7	59692.1	31419.5	66750.6	19773.4
NL2Plan	98	2	1	5	20	0	0	2.8	265195.9	203457.1	46588.9	33257.3
ISR-LLM	–	–	60	46	70	20	66.7	22.2	–	–	–	–
COT	–	–	87	19	25	0	46.7	0	3663.8	3944.0	4920.7	5009.0

Table 8: Performance of gpt-oss-120b across PDDLCoder and our adaptations of previous LLM-Formalizers (VML_PDDL and NL2Plan) and LLM-Planners (ISR-LLM and COT) on the NL-pddlgyim test set

²OpenRouter: available under <https://openrouter.ai/>

Approach	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
PDDLCodeR	45	1	49	11	30	5	6.7	5.6	129916.1	251746.6	37574.6	40290.6
VML_PDDL	106	0	0	0	0	0	0	0	118813.5	43678.3	117951.1	11979.3
NL2Plan	104	1	1	0	0	0	0	0	99211.5	367813.5	38490.0	139312.0
ISR-LLM	–	–	–	–	–	–	–	–	–	–	–	–
COT	–	–	101	5	5	0	13.3	0	4113.5	5231.2	14464.9	8999.2

Table 9: Performance of Qwen 3.6 35b A3b across PDDLCodeR and our adaptations of previous LLM-Formalizers (VML_PDDL and NL2Plan) and LLM-Planners (ISR-LLM and COT) on the NL-pddlgyM test set. Note, that results with ISR-LLM could not be generated as the prompting approach used by ISR-LLM, which includes intermediate system prompts, is not supported by this model.

Approach	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
PDDLCodeR	66	17	22	1	5	0	0	0	762909.5	721951.2	35162.3	28381.5
VML_PDDL	105	0	1	0	0	0	0	0	96135.3	40211.3	107913.0	12473.0
NL2Plan	106	0	0	0	0	0	0	0	217176.2	430568.5	88899.5	168316.3
ISR-LLM	–	–	106	0	0	0	0	0	–	–	–	–
COT	–	–	105	1	0	0	3.3	0	3654.9	4809.9	8620.3	9011.0

Table 10: Performance of GLM 4.7 Flash across PDDLCodeR and our adaptations of previous LLM-Formalizers (VML_PDDL and NL2Plan) and LLM-Planners (ISR-LLM and COT) on the NL-pddlgyM test set.

Approach	Outcome Counts				Success Rates (%)				Token Statistics			
	L0	L1	L2	L3	El	Ha	RP	Sa	N-In	S-In	N-Out	S-Out
PDDLCodeR	99	3	4	0	0	0	0	0	521045.1	154822.6	13961.4	6312.5
VML_PDDL	106	0	0	0	0	0	0	0	48656.2	31920.6	32228.2	13921.2
NL2Plan	76	11	14	5	10	5	6.7	0	164695.2	89894.1	15800.3	9109.9
ISR-LLM	–	–	101	5	10	5	3.3	2.8	–	–	–	–
COT	–	–	101	5	15	0	6.7	0	2814.9	2199.8	893.2	517.6

Table 11: Performance of Llama 4 Scout 17b 16e across PDDLCodeR and our adaptations of previous LLM-Formalizers (VML_PDDL and NL2Plan) and LLM-Planners (ISR-LLM and COT) on the NL-pddlgyM test set.

C Example of a Successful PDDLCoder Run

The following presents a successful PDDLCoder execution trace for a problem in the Ring and Peg domain. The task starts out with the following D_{NL} and P_{NL}

D_{NL} and P_{NL}

Domain:

Domain description

A robot arm is tasked to sort a set of colored rings on a set of pegs of the same colors. The arm can be used to pick up a ring from a peg, move a ring to another peg and place the ring on the other peg. Rings are already placed on pegs and can only be placed on pegs. To pick up a ring at a peg, the robot first has to move to the pegs position.

The actions available to the robot are:

- move
 - Moves the robot to a specified position
- pick
 - Closes the robot arms gripper at the current position
- place
 - Opens the robot arms gripper at the current position

Problem:

Problem description

There are 5 colored pegs with the following colors: red, green, blue, pink, yellow. The pegs are all named in the format < color>_peg. Additionally there are 4 colored rings: red, green, yellow and pink. Similarly to the pegs, the rings are named < color>_ring.

The robot arm starts out in a default position, while the rings start in the following positions:

- red_ring – red_peg
- green_ring – yellow_peg
- yellow_ring – blue_peg
- pink_ring – green_peg

The goal is to transfer all rings to the pegs of their color.

PDDLCoder then initially creates the following files, which are verified to be syntactically correct after creation.

D

```
(define (domain ring-sorting)
  (:requirements :strips :typing :negative-preconditions :equality)
  (:types
    location
    peg – location
    ring
  )
  (:predicates
    (robot_at ?l – location)
    (hand_empty)
    (holding ?r – ring)
    (on_peg ?r – ring ?p – peg)
  )
  (:action move
    :parameters (?from ?to – location)
    :precondition (and (robot_at ?from) (not (= ?from ?to)))
    :effect (and (robot_at ?to) (not (robot_at ?from)))
  )
  (:action pick
```

```

:parameters (?r – ring ?p – peg)
:precondition (and (robot_at ?p) (hand_empty) (on_peg ?r ?p))
:effect (and (holding ?r) (not (hand_empty)) (not (on_peg ?r ?p)))
)
(:action place
:parameters (?r – ring ?p – peg)
:precondition (and (robot_at ?p) (holding ?r))
:effect (and (on_peg ?r ?p) (hand_empty) (not (holding ?r))))
)
)

```

P

```

(define (problem sort-rings)
  (:domain ring-sorting)
  (:objects
    default_pos – location
    red_peg green_peg blue_peg pink_peg yellow_peg – peg
    red_ring green_ring yellow_ring pink_ring – ring
  )
  (:init
    (robot_at default_pos)
    (hand_empty)
    (on_peg red_ring red_peg)
    (on_peg green_ring yellow_peg)
    (on_peg yellow_ring blue_peg)
    (on_peg pink_ring green_peg)
  )
  (:goal
    (and
      (on_peg red_ring red_peg)
      (on_peg green_ring green_peg)
      (on_peg yellow_ring yellow_peg)
      (on_peg pink_ring pink_peg)
    )
  )
)
)

```

The LLM then goes on to call the tool for plan generation, which generates the following plan.

π

```

(move default_pos blue_peg)
(pick yellow_ring blue_peg)
(move blue_peg green_peg)
(move green_peg yellow_peg)
(place yellow_ring yellow_peg)
(pick green_ring yellow_peg)
(move yellow_peg green_peg)
(place green_ring green_peg)
(pick pink_ring green_peg)
(move green_peg pink_peg)
(place pink_ring pink_peg)
; cost = 11 (unit cost)

```

To verify that π is correctly aligned with D_{NL} and P_{NL} , the LLM then goes on to call the PlanAgent, which returns the following feedback.

Plan Feedback

The plan is logically flawed and physically infeasible. After moving to yellow_peg and placing the yellow_ring, the next action (pick) at the same peg would pick the topmost ring, which is the just-placed yellow_ring, not the green_ring that needs to be moved. This causes the subsequent moves to place the yellow_ring incorrectly on green_peg and then pink_peg. The plan cannot achieve the goal because it lacks a mechanism to select a specific ring from a peg with multiple rings. Additionally, the detour to green_peg before yellow_peg is unnecessary but not the main issue.

In response, the LLM then generates the following output and goes on to apply the fix to D and P , which are shown in their updated form below. While the noted fix is not the only valid approach to the problem, as it avoids the infeasibility of picking a lower ring from a stack by simply disallowing ring stacking, this approach still results in an applicable plan for the given problem.

Thought

The plan feedback indicates that the current domain allows picking any ring from a peg regardless of stack order, which is physically infeasible. To fix this, we should enforce that each peg can hold at most one ring. This avoids stacking issues and matches the problem well, as there are 5 pegs and only 4 rings, leaving one peg empty. We need to add a predicate '(peg_empty ?p - peg)' and modify the 'place' action to require the destination peg to be empty, and the 'pick' action to make the peg empty after picking. Also update the problem file with initial and goal 'peg_empty' states. We'll edit the domain and problem files accordingly.

Updated D

```
(define (domain ring-sorting)
  (:requirements :strips :typing :negative-preconditions :equality)
  (:types
    location
    peg - location
    ring
  )
  (:predicates
    (robot_at ?l - location)
    (hand_empty)
    (holding ?r - ring)
    (on_peg ?r - ring ?p - peg)
    (peg_empty ?p - peg)
  )
  (:action move
    :parameters (?from ?to - location)
    :precondition (and (robot_at ?from) (not (= ?from ?to)))
    :effect (and (robot_at ?to) (not (robot_at ?from)))
  )
  (:action pick
    :parameters (?r - ring ?p - peg)
    :precondition (and (robot_at ?p) (hand_empty) (on_peg ?r ?p))
    :effect (and (holding ?r) (not (hand_empty)) (not (on_peg ?r ?p)) (peg_empty ?p))
  )
  (:action place
    :parameters (?r - ring ?p - peg)
    :precondition (and (robot_at ?p) (holding ?r) (peg_empty ?p))
    :effect (and (on_peg ?r ?p) (hand_empty) (not (holding ?r)) (not (peg_empty ?p)))
  )
)
```

Updated P

```

(define (problem sort-rings)
  (:domain ring-sorting)
  (:objects
    default_pos - location
    red_peg green_peg blue_peg pink_peg yellow_peg - peg
    red_ring green_ring yellow_ring pink_ring - ring
  )
  (:init
    (robot_at default_pos)
    (hand_empty)
    (peg_empty pink_peg)
    (on_peg red_ring red_peg)
    (on_peg green_ring yellow_peg)
    (on_peg yellow_ring blue_peg)
    (on_peg pink_ring green_peg)
  )
  (:goal
    (and
      (on_peg red_ring red_peg)
      (on_peg green_ring green_peg)
      (on_peg yellow_ring yellow_peg)
      (on_peg pink_ring pink_peg)
    )
  )
)

```

As both files are syntactically correct and solvable, they yield the following π generated by the planning tool.

Updated π

```

(move default_pos blue_peg)
(move blue_peg green_peg)
(pick pink_ring green_peg)
(move green_peg blue_peg)
(move blue_peg pink_peg)
(place pink_ring pink_peg)
(move pink_peg blue_peg)
(move blue_peg yellow_peg)
(pick green_ring yellow_peg)
(move yellow_peg green_peg)
(place green_ring green_peg)
(move green_peg blue_peg)
(pick yellow_ring blue_peg)
(move blue_peg yellow_peg)
(place yellow_ring yellow_peg)
; cost = 15 (unit cost)

```

To verify that the generated π is properly aligned with the NL descriptions, the LLM then calls the PlanAgent once more and finishes execution after 12 iterations, as the PlanAgent states that the generated π is properly aligned.

Plan Feedback on Updated π

The plan is logically and physically feasible. It correctly moves the pink ring from green peg to pink peg, the green ring from yellow peg to green peg, and the yellow ring from blue peg to yellow peg. All picks and places occur at pegs that contain the intended ring or are empty, respectively. The sequence of moves allows the arm to reach the necessary positions while holding rings appropriately. No logical or physical constraints are violated.

After the execution has succeeded, D and P are externally checked for syntactic validity, π is regenerated, and the MapAgent finally generates the following applicable π_m .

π_m

```
(move blue_peg)
(move green_peg)
(pick)
(move blue_peg)
(move pink_peg)
(place)
(move blue_peg)
(move yellow_peg)
(pick)
(move green_peg)
(place)
(move blue_peg)
(pick)
(move yellow_peg)
(place)
```

D PDDLCoder Prompts

PDDLCoder Prompt

Task Context

You are an expert at generating PDDL domain and problem files. You will be asked to generate PDDL files for a task given in natural language.

Make sure to adhere to the naming of objects, entities and actions given in the prompt. There are no other objects in the domain and the robot is only able to carry out the given actions. No other actions than the ones explicitly listed for a task are allowed in the domain and plan.

Additionally, make sure your output is complete and can be used by a solver to generate a plan once both domain and problem files have been generated. This means including a full definition of the actions, predicates and other PDDL attributes you deem necessary to complete the task.

After creating the files, verify that they do not contain syntax mistakes. If they do contain mistakes, edit the files to fix them. Additionally, ensure that the task defined in the PDDL files is actually solvable using the Fast Downward planning system. Before a plan can be generated, ensure that the translation layer of the Fast Downward planning system runs without reporting any further issues. Once a plan has been generated ensure it is actually physically and logically feasible for the given task by getting feedback from the `get_plan_feedback` function. ****Always**** use and incorporate the feedback from the `get_plan_feedback` function before completing your task. The generated plan needs to satisfy the described physical domain, not only the PDDL files you generated.

All PDDL files have to adhere to the PDDL 1.2 standard. Do not use constants or derived predicates. You may use the following requirements: strips, disjunctive-preconditions, conditional-effects, negative-preconditions

MapAgent Prompt

Role

You are an expert of mapping a plan of PDDL actions to a plan with a different action schema and object naming.

You will be given the original plan of PDDL actions, the corresponding PDDL domain with the definitions of the plans actions, a definition of the actions that the plan sequence should be mapped to and lastly a list of the object names available in the environment.

Make sure to fit the actions parameters and object names to the semantically closest parameters of the new action schema.

If the new action schema does not contain any parameters, you do not need to map any parameters or object names.

If the plan contains actions not given in the schema, leave those actions in the plan as is.

Note, that the same MapAgent prompt is being used in our adaptations of VML_PDDL and NL2Plan when mapping the generated plans to their simulation-ready form.

PlanAgent Prompt

Role

You are an expert at providing feedback for the feasibility and applicability of a given plan. You will be provided with a description of the plan that should be solved, the plan and the schema of the involved actions.

****Your focuses are**:**

- Does the sequence of actions logically lead to the desired outcomes?
- Is the sequence of actions physically and logically feasible?

The plan will need to be carried out in the real world and should therefore respect real logical and physical constraints.

Do not make suggestions apart from the focuses noted above.

****Important**:**

- The plan does not need to be optimal as long as it solves the given task. Do not provide feedback on redundant moves.
- Ensure that the plan only uses the actions given in the task description.
- Be specific in your critique and provide suggestions to fix logical/physical flaws.
- Do not try to provide a plan yourself.
- Provide actional feedback on why a plan is feasible/unfeasible.
- The cost annotations at the end of the file are an artifact and have no relevance for your feedback.
- BE CONCISE.

E Rigid PDDLCoder Prompts

Rigid PDDLCoder Prompt

Task Context

You are an expert at generating PDDL domain and problem files. You will be asked to generate PDDL files for a task given in natural language. You will first be asked to generate the domain file for the given task.

Make sure to adhere to the naming of objects and actions given in the prompt.

Additionally, make sure your output is complete and can be used by a solver to generate a plan once both domain and problem files have been generated. This means including a full definition of the actions, predicates and other PDDL attributes you deem necessary to complete the task.

All PDDL files have to adhere to the PDDL 1.2 standard. If possible, refrain from defining :constants in the domain. It should be possible to define multiple problems with different objects for the same domain.

Answer ****only**** with PDDL as output. The result will directly be used in a PDDL planning system and should therefore be syntactically and semantically sound.

Syntax Error Prompt

Task Context

You are an expert at fixing PDDL files.

You will be given PDDL domain or problem files, which contain syntax errors.

In addition to the PDDL files, you will also be presented with the list of errors in the given file.

The errors will present the original line from the PDDL file and the corresponding error message.

You will need to fix the syntax mistakes and return the fixed file.

Answer ****only**** with PDDL as output. The result will directly be used in a PDDL planning system and should therefore be syntactically and semantically sound.

All PDDL files have to adhere to the PDDL 1.2 standard.

Translation Error Prompt**## Task Context**

You are an expert at fixing syntactically invalid PDDL domain and problem files. You will be asked to fix and improve a given pair of PDDL domain and problem files that currently contain syntax mistakes. You will be given information on which of the two you should fix, as only one of the two files can be changed at a time.

All PDDL files have to adhere to the PDDL 1.2 standard.

Answer ****only**** with PDDL as output. The result will directly be used in a PDDL planning system and should therefore be syntactically and semantically sound.

Planning Error Prompt**## Task Context**

You are an expert at fixing unsolvable PDDL domain and problem files. You will be asked to fix and improve a given pair of PDDL domain and problem files that are currently unsolvable. You will be prompted with which of the two you should be improving, as only one of the two files should be changed at a time.

Answer ****only**** with PDDL as output. The result will directly be used in a PDDL planning system and should therefore be syntactically and semantically sound.

All PDDL files have to adhere to the PDDL 1.2 standard.

F COT Prompts**COT Prompt****## Task Context**

You are an expert at generating plans for solving a given problem.

Make sure to adhere to the naming of objects, entities and actions given in the prompt. There are no other objects in the domain and the robot is only able to carry out the given actions. No other actions than the ones explicitly listed for a task are allowed in the domain and plan.

Additionally, make sure your output is complete and actually solves the given problem. Note, that for the plan to be correct, each of the actions and their parameters in the plan need to be written to a new line in the plan, surrounded by parantheses. So for an action 'action' with parameters 'p1', 'p2' the resulting step in the plan would be '(action p1 p2)'.

MapAgent Prompt**# Role**

You are an expert of mapping a plan of PDDL actions to a plan with a different action schema and object naming.

You will be given the original plan of PDDL actions, a definition of the actions that the plan sequence should be mapped to and lastly a list of the object names available in the environment.

Make sure to fit the actions parameters and object names to the semantically closest parameters of the new action schema.

If the new action schema does not contain any parameters, you do not need to map any parameters or object names.

If the plan contains actions not given in the schema, leave those actions in the plan as is.

Note, that the MapAgent prompt that is being used for the COT implementation is the same for the final plan mapping that is being used in our adaptation of ISR-LLM.

G Adapted Related Work Prompts

To run VML_PDDL with the remotely hosted DeepSeek model, we adapted the original approach to run with LLM-based grading instead of the original log-likelihood based grading system. Following is the prompt used for this grading step. All other prompts that are used in the related work are the same as in the original implementations.

LLM-Based Grading of Generated Domains for VML_PDDL

Score the following PDDL domain on how well it captures the semantics of the NL description (0–100). Return only the number.

NL: {nl_description}

PDDL Domain:
{domain}

Score:

LLM-Based Grading of Generated Problems for VML_PDDL

Score the following PDDL problem on how well it captures the semantics of the NL description and is consistent with the given domain (0–100). Return only the number.

NL: {nl_description}

Domain:
{domain}

Problem:
{problem}

Score:

H NL-pddlgyM Test Set Domain and Problem Descriptions**Elevator Domain Description****## Domain Description**

A robot is tasked with driving an elevator to pick up people and drive them to their desired floor.

The available actions are:

- down
 - Moves the elevator down 1 floor
- up
 - Moves the elevator up 1 floor
- board
 - Lets the passenger board the elevator
- depart
 - Lets the passenger depart the elevator

Exemplary Elevator Problem Description**## Problem Description**

There are six floors (f0 – f5) and three passengers (p0 – p2). The floors are arranged in the following order (bottom to top): f0, f1, f2, f3, f4, f5. The lift is currently at floor f0. The passenger p0 is at floor f1, p1 is at f3 and p2 is at f5. The goal is to transfer the passenger p0 to floor f4, p1 to f1 and p2 to f1.

Hanoi Domain Description

Domain description

A robot is tasked with stacking a set of rings of different sizes on a set of pegs such that the rings are stacked on one peg with the largest ring of the bottom with each subsequent ring being the next smaller available ring.

The actions available to the robot are:

- move
 - Moves a ring from one peg to the other.

Exemplary Hanoi Problem Description

Problem description

There are 3 pegs (peg1 – peg3) and 4 rings (d1 – d4). The rings have the following order according to their size (largest to smallest): d4,d3,d2,d1. peg2 and peg3 have no rings stacked on them. peg1 has rings stacked on it in the following order (bottom to top): d4,d3,d2,d1. The rings should be stacked from largest to smallest on peg peg3.

Satellite Domain Description

Domain description

A robot is tasked with controlling a set of satellites to capture images. Each satellite can have different instruments with different capabilities on board.

The actions available to the robot are:

- turn_to
 - Points a satellite in a specific direction.
- switch_on
 - Turns on a specific instrument on the satellite. Once an instrument is switched on on a satellite, all power is consumed by this instrument and the power only becomes available again once the instrument is turned off.
- switch_off
 - Turns off a specific instrument on the satellite.
- calibrate
 - Calibrates an instrument for a specific target direction.
- take_image
 - Takes an image in a specific mode of the current direction of the satellite using one a calibrated instrument.

Exemplary Satellite Problem Description

Problem description

There are two satellites (satellite0, satellite1), four instruments (instrument0 – instrument3), three different image modes (image1, spectograph2, infrared0) and eight directions (Star0 – Star4, Phenomenon5 – Phenomenon7). The satellites satellite0 and satellite1 have power available. The satellite satellite0 carries the instruments instrument0, instrument1 and instrument2 and satellite1 carries instrument3. The instrument instrument0 support the following modes: spectograph2, infrared0. instrument1 supports image1. instrument2 supports infrared0 and image1. instrument3 supports spectograph2, infrared0 and image1. instrument0 is currently calibrated for the Star1, instrument1 for Star2, instrument2 for Star0 and instrument3 for Star0. The satellite satellite0 is currently pointing in the direction of Star4 and satellite1 is pointing to Star0. The goal is to capture an image of Star3 with infrared0, of Star4 with spectograph2, of Phenomenon5 with spectograph2, of Phenomenon7 with spectograph2 and for satellite0 to point to Phenomenon5.